

Beyond Machine Learning

Models, Inference Engines & AI Servers in 2026

A presentation guide to how trained AI models are actually served to thousands or millions of users.

Core idea

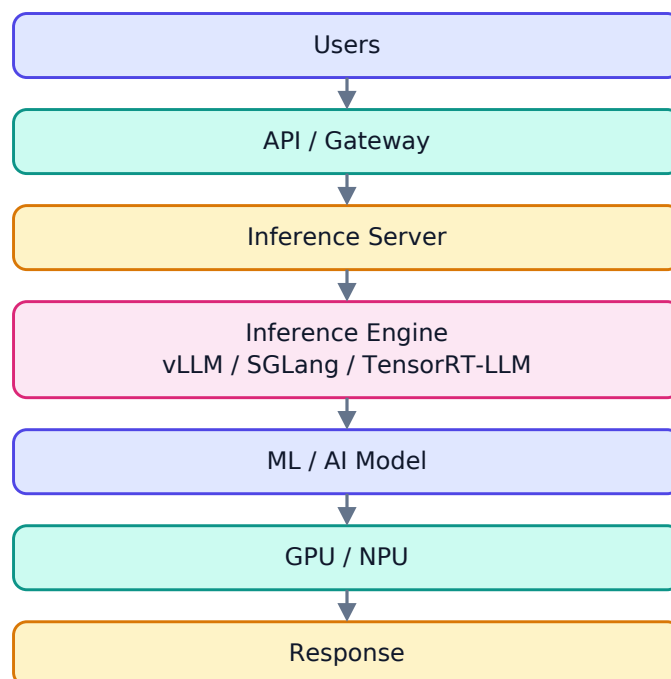
Traditional ML teaching usually focuses on:

Dataset → Training → Model → Prediction

But in a real company, training the model is only part of the problem. Once the model is trained, you have to answer:

How do thousands or millions of users actually use this model at the same time?

That is where **model serving and inference infrastructure** come in. A production AI system therefore looks more like this:



The interesting part is everything happening **between the API and the GPU**.

What is covered

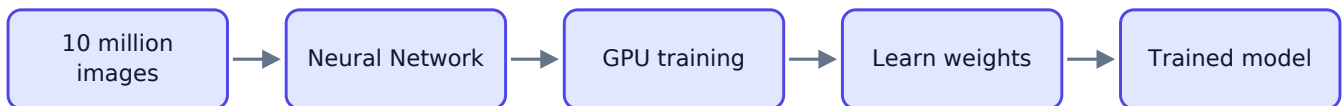
- | | |
|---------------------------------------|---|
| 1 Training vs Inference | 12 vLLM example |
| 2 What is an Inference Server? | 13 Triton Inference Server |
| 3 Why 2026 is different | 14 Traditional ML still matters |
| 4 KV Cache | 15 A real-world company architecture |
| 5 Continuous Batching | 16 The three things companies optimize |
| 6 Speculative Decoding | 17 Latency vs throughput |
| 7 Quantization | 18 Local AI vs Cloud AI |
| 8 Mixture of Experts | 19 Edge AI |
| 9 NVIDIA Dynamo | 20 The evolution timeline |
| 10 Prefill vs Decode | 21 The final architecture |
| 11 The major inference engines | + Conclusion |

1 First: Training vs Inference

This distinction should be your opening slide.

Training

Training means teaching the model. For example:



This can take hours, days or even weeks.

Inference

Inference means **using the trained model**.



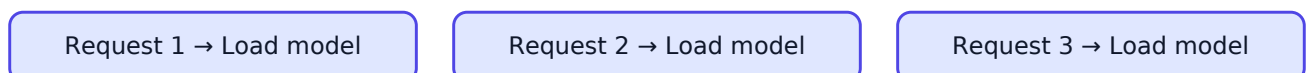
And unlike training, inference happens **every time a user interacts with the AI**.

So a company might train a model once, but serve **millions or billions of inference requests** afterward. That's why inference efficiency has become such a major engineering problem.

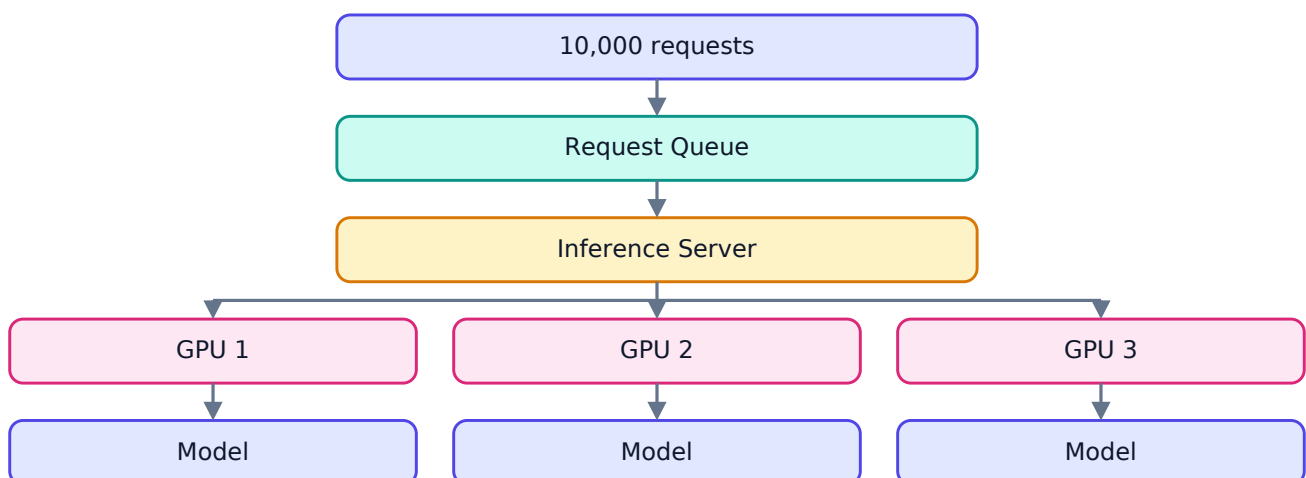
2 What is an Inference Server?

Think of an inference server as a **traffic controller for AI models**.

Suppose 1 person sends: *"Write Python code for sorting."* Easy. But now imagine **10,000 people simultaneously asking an AI questions**. You cannot simply do this:



That would waste enormous amounts of GPU memory and computation. Instead:



The server decides:

- which GPU handles the request
- when it should run
- which requests can run together
- how GPU memory is allocated
- how model states are cached
- how requests are distributed
- when additional GPUs are needed

That's **AI infrastructure engineering**.

3 Why 2026 is different

The model itself is becoming only one piece of the system. Modern models can be:

- extremely large
- multimodal
- mixture-of-experts
- long-context
- reasoning-oriented
- capable of tool calling
- capable of autonomous agent workflows

That creates a new problem: **How do we serve these models efficiently?**

Current serving stacks are therefore optimizing things like:

latency + throughput + GPU memory + cost + reliability

For example, current vLLM supports PagedAttention, continuous batching, prefix caching, speculative decoding, quantization and distributed inference. [1]

These are the concepts worth spending the most time explaining.

A. KV Cache

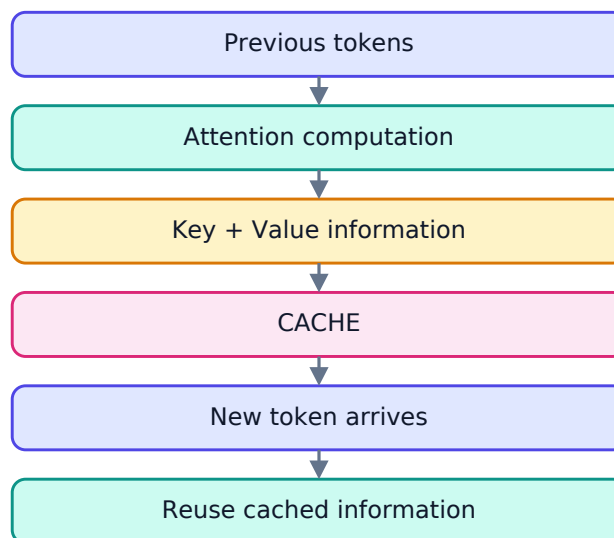
This sounds complicated but the basic idea is simple. Imagine you're chatting with an AI:

User: My name is Parth.

AI: Nice to meet you, Parth.

User: What is my name?

The model has already processed "My name is Parth." It shouldn't need to completely recompute everything from scratch every time. The **KV cache** stores useful intermediate attention information from previous tokens. Simplified:



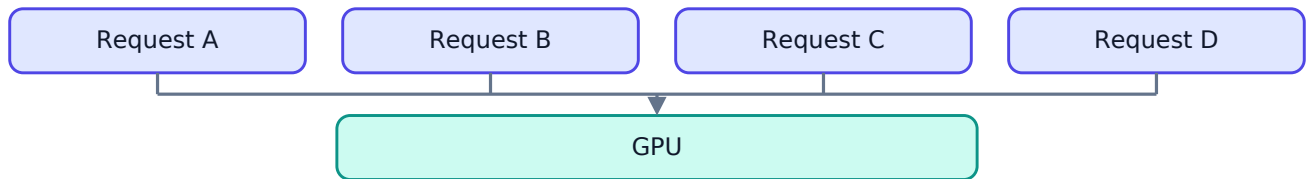
Why it matters

For long conversations, KV cache can consume **a huge amount of GPU memory**. So modern inference systems have to manage this memory extremely carefully. This is one reason systems such as vLLM use specialized memory management approaches such as **PagedAttention**. [1]

5

Continuous Batching

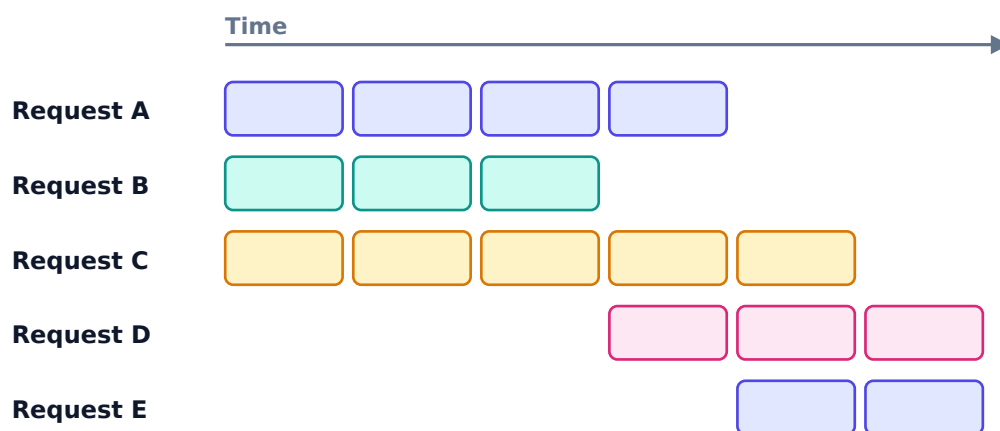
This is another excellent concept for your presentation. Traditional batching:



The problem? What happens when A finishes quickly but D takes much longer? You don't want the GPU sitting around doing nothing.

Continuous batching

New requests can enter while other requests are still being processed.



The server continuously manages the workload. This keeps expensive GPUs busy. vLLM explicitly supports continuous batching for incoming requests. [1]

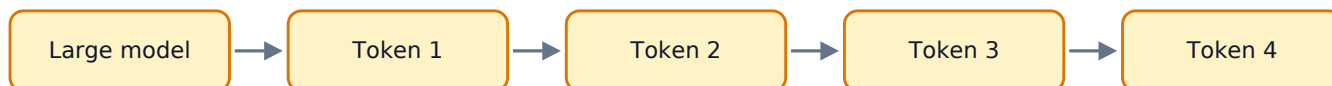
Why companies care

More requests processed per GPU means: **higher throughput → better hardware utilization → potentially lower cost per request**

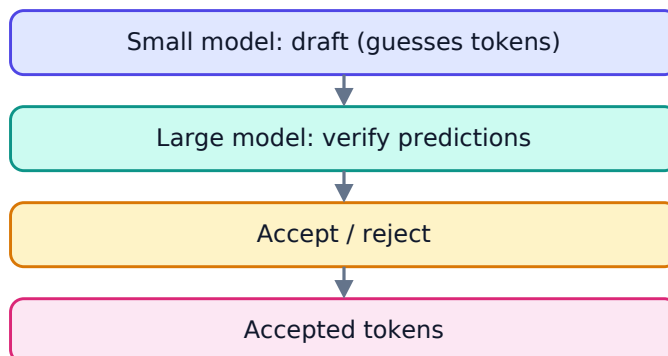
6 Speculative Decoding

This one will make your presentation sound considerably more advanced. Imagine you have a **large model** (a 70B parameter model: very capable, but expensive) and a **small model** (a 1B parameter model: much faster).

Instead of asking the large model to generate every token individually:



you can use the smaller model to **guess several tokens**. Conceptually:

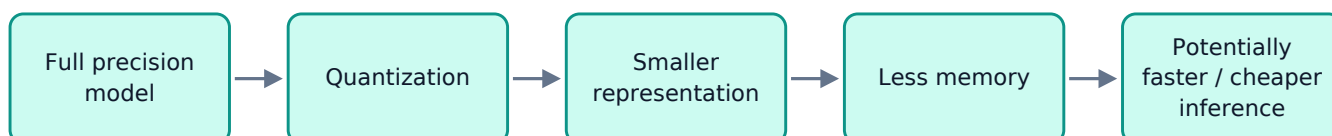


If many guesses are correct, you can generate output faster. Modern serving systems including vLLM support multiple forms of speculative decoding. [1]

7 Quantization

Make the model smaller and cheaper to run while trying to preserve useful accuracy.

A model might originally use **FP16**, but can potentially use lower-precision representations such as **INT8**, **INT4** or **FP8**. Conceptually:



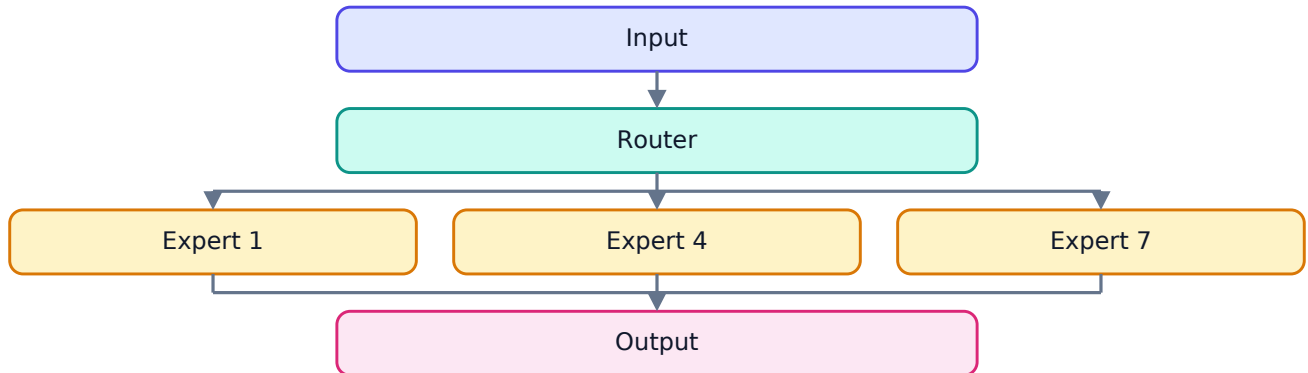
Modern vLLM supports a broad range of quantization formats, including FP8, MXFP variants, INT8, INT4, GPTQ and AWQ. [1]

This becomes particularly interesting when trying to run AI on:

- laptops
- phones
- edge devices
- smaller GPUs
- enterprise servers

8 Mixture of Experts

Now we get into the really interesting architecture. Imagine a company has an enormous AI model. Instead of every input using **every part of the model**, the architecture contains multiple specialized "experts."



For one request, only some experts are activated. For another request (Input B), the router might select **Expert 2 + Expert 5**. This is **Mixture of Experts (MoE)**.

The challenge is that those experts may be distributed across many GPUs. So now the inference server has to figure out: *which GPU has the expert I need?*

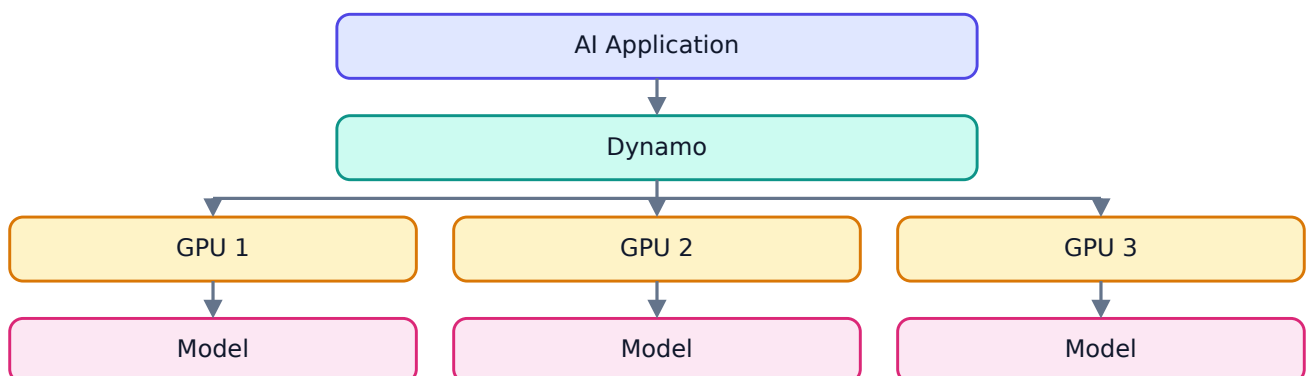
That turns AI serving into a **distributed systems problem**. This is exactly where modern infrastructure such as NVIDIA Dynamo becomes interesting.

9 NVIDIA Dynamo

This is probably the **standout section** to add to your presentation.

NVIDIA introduced **Dynamo 1.0 in March 2026** as an open-source inference-serving framework designed for production-scale generative AI. NVIDIA describes it as an infrastructure layer for distributed inference. [2]

The important idea: you don't necessarily have one giant server. You can have:



Dynamo can work with inference engines including:

- **vLLM**
- **SGLang**
- **TensorRT-LLM**

and is designed for distributed, multi-node inference. [3]

This is a major conceptual upgrade from "Run an ML model on a server." to:

"Build infrastructure capable of serving an AI model across an entire GPU cluster."

10 Prefill vs Decode

This is another **very 2026** concept. LLM generation basically has two important phases.

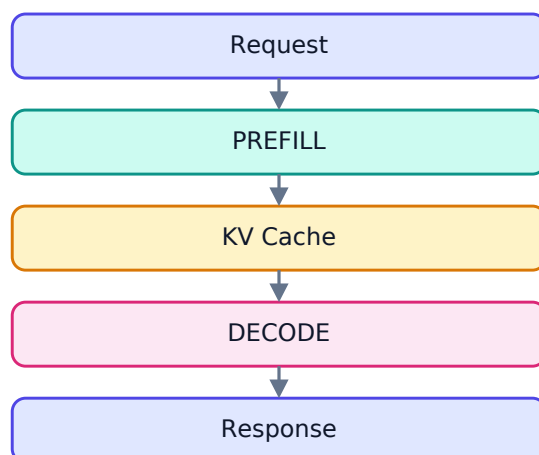
Prefill

The model processes the input prompt, for example: "Explain machine learning in simple words".

Decode

The model generates the answer token by token: Machine → learning → is → ...

These workloads have different characteristics. Modern distributed inference systems can therefore separate them:



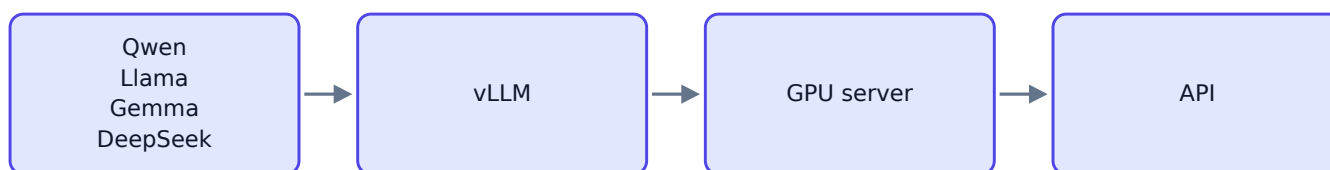
This is called **disaggregated inference**. NVIDIA Dynamo explicitly uses disaggregation of inference phases as part of its architecture. [3]

11 The major inference engines

Now you can introduce the actual technologies.

Technology	Main idea	Typical role
vLLM	High-throughput LLM serving	LLM APIs
SGLang	Efficient structured LLM programs	Agents/reasoning
TensorRT-LLM	NVIDIA GPU optimization	High-performance inference
Triton	General model serving	Production ML platform
llama.cpp	Efficient local inference	Laptops/edge
MLX	Apple Silicon ML	Local Mac AI
Ollama	Easy local model management	Local AI development
Dynamo	Distributed inference orchestration	Large AI clusters

One important distinction: **vLLM** $\neq$ **model**. vLLM isn't an AI model. You might have:



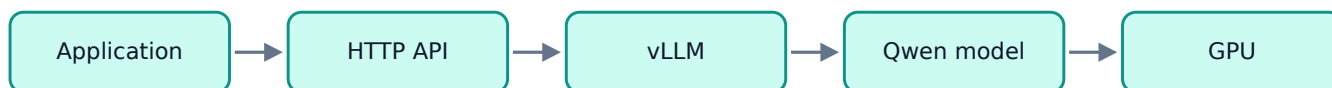
- The **model provides intelligence**.
- The **inference engine provides efficient execution**.
- The **server exposes that intelligence to applications**.

12 vLLM example

This is a great practical demonstration. You can actually start a model server using something like:

```
vllm serve Qwen/Qwen2.5-1.5B-Instruct
```

vLLM then exposes an HTTP API, including OpenAI-compatible endpoints. [4] Your application can then communicate with:



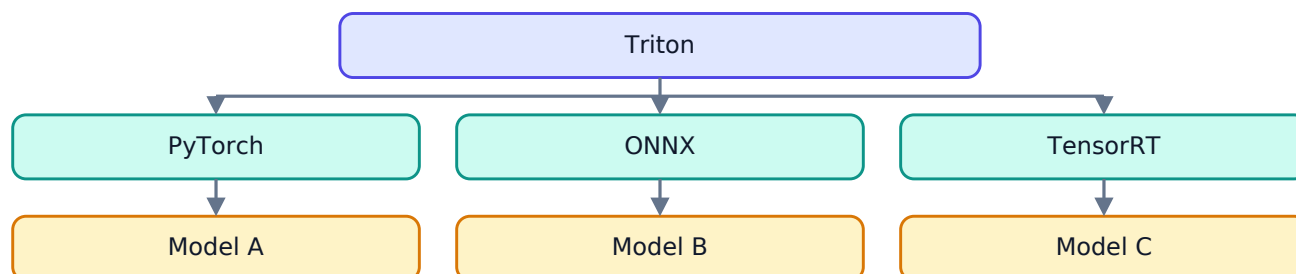
So your Python application doesn't necessarily need to directly manage the model. It just sends:

```
{
  "model": "Qwen/Qwen2.5-1.5B-Instruct",
  "prompt": "Explain AI"
}
```

and receives the generated output. That's **model serving**.

13 Triton Inference Server

Triton is broader than something like vLLM. It can serve different kinds of ML workloads rather than being purely an LLM-serving engine. Think:



NVIDIA's current 26.08 Triton release includes CUDA 13.4.1 and TensorRT 11.2.1.2, while its compatibility stack includes modern vLLM and TensorRT-LLM components. [5]

So Triton is useful when a company has **different types of models** to serve.

14 Traditional ML still matters

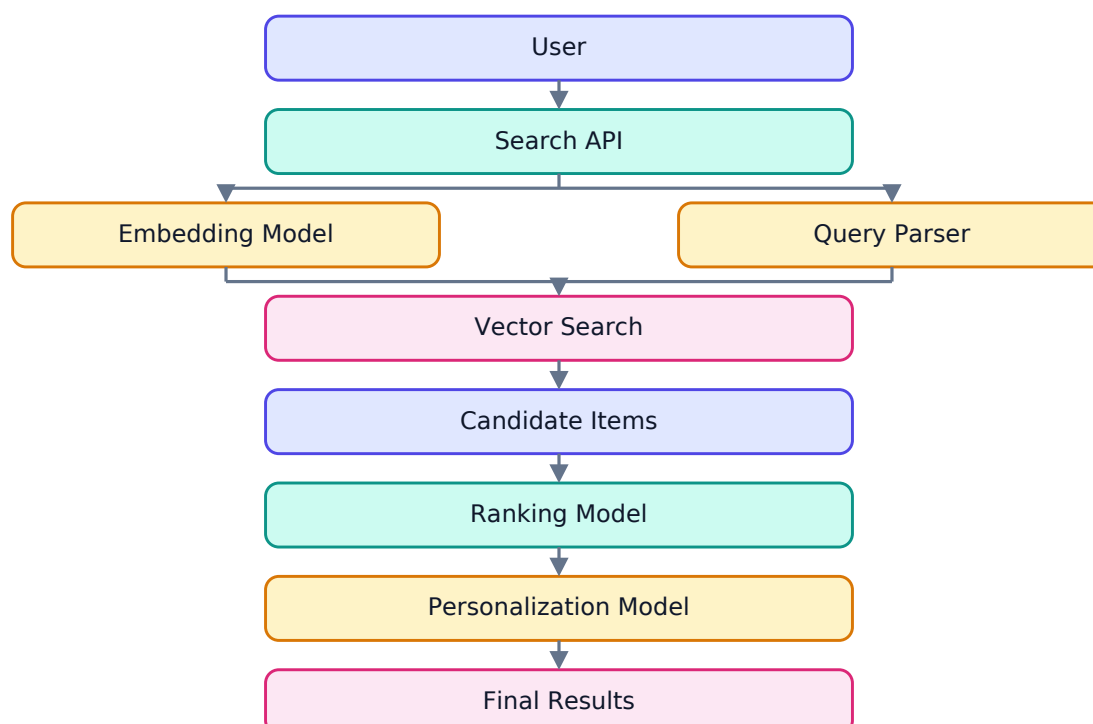
This is an important point. Don't make your presentation sound like: *"LLMs replaced everything."* They haven't. A company could have:

Use case	Model type
Fraud detection	XGBoost
Recommendation	Two-Tower / ranking model
Image inspection	YOLO / Vision Transformer
Demand forecasting	Time-series model
Customer chatbot	LLM

And all of these can exist inside the **same company's production ML infrastructure**. So modern AI infrastructure needs to serve both **classical ML** and **generative AI**.

15 A real-world company architecture

Imagine an e-commerce company. A customer searches: *"Black running shoes under ₹5000"*. The backend could involve multiple models.



Then perhaps an LLM generates: *"Here are 5 running shoes..."*

Now you have **Embeddings + Search + Ranking + Recommendation + LLM** all working together. That is much closer to how modern AI systems are architected.

16 The three things companies optimize

This would make a very good slide.

Latency

How quickly does the user receive the response?

Example:

Request → 200 ms → Response

For chat applications, another useful metric is **time to first token (TTFT)**.

Throughput

How many requests/tokens can the infrastructure process?

Example:

1 GPU → 100 requests/sec

Higher throughput can improve utilization.

Cost

How much does every request cost?

GPU cost + electricity + infrastructure + memory + networking + storage → **inference cost**.

If an optimization reduces the amount of GPU capacity needed, that can have a direct infrastructure-cost impact.

17 Latency vs throughput

Here's a nice example for your presentation. Suppose:

System A	System B
Response: 500 ms	Response: 150 ms
Throughput: 100 requests/sec	Throughput: 300 requests/sec

System B isn't simply "a better model." It could be using:

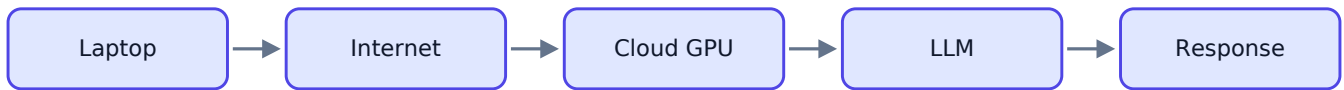
- better batching
- better GPU utilization
- quantization
- optimized kernels
- KV-cache management
- better request scheduling

The model might even be identical. That's the key insight behind your whole presentation.

18 Local AI vs Cloud AI

This is another very current angle.

Cloud



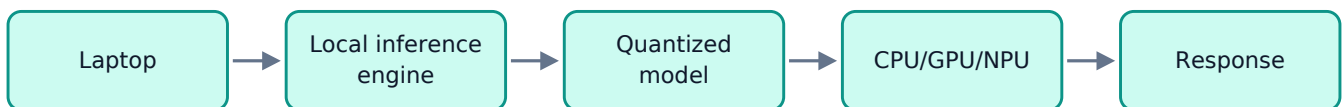
Advantages:

- huge models
- powerful GPUs
- centralized infrastructure
- easy scaling

But:

- network latency
- privacy considerations
- API/infrastructure cost
- dependency on connectivity

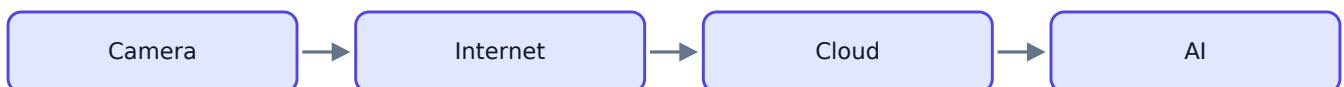
Local AI



Technologies such as **llama.cpp** and **MLX** are relevant here. This means your laptop itself can become an AI inference machine.

19 Edge AI

Take the same idea one step further. Instead of:



you can have:



For example:



No need to send every video frame to the cloud. This is useful where:

- latency matters
- connectivity is unreliable
- privacy matters
- bandwidth is expensive

20 The evolution you can show

This could literally be your **timeline slide** (read down the left column, then the right):

1 Traditional ML	8 Quantization
2 Model Training	9 Speculative Decoding
3 Model API	10 MoE
4 GPU Inference	11 Distributed Inference
5 Batching	12 AI Agents
6 LLM Serving	13 AI Inference Infrastructure
7 KV Cache	

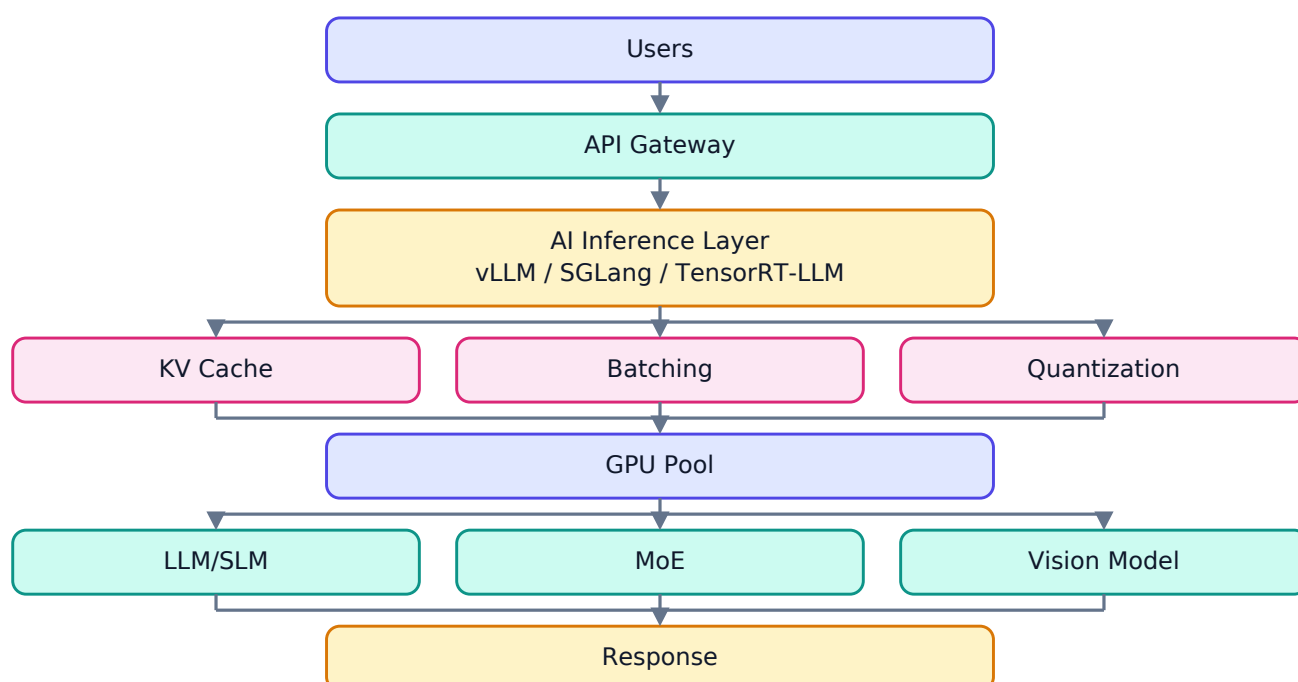
The big shift is:

Model-centric AI → System-centric AI

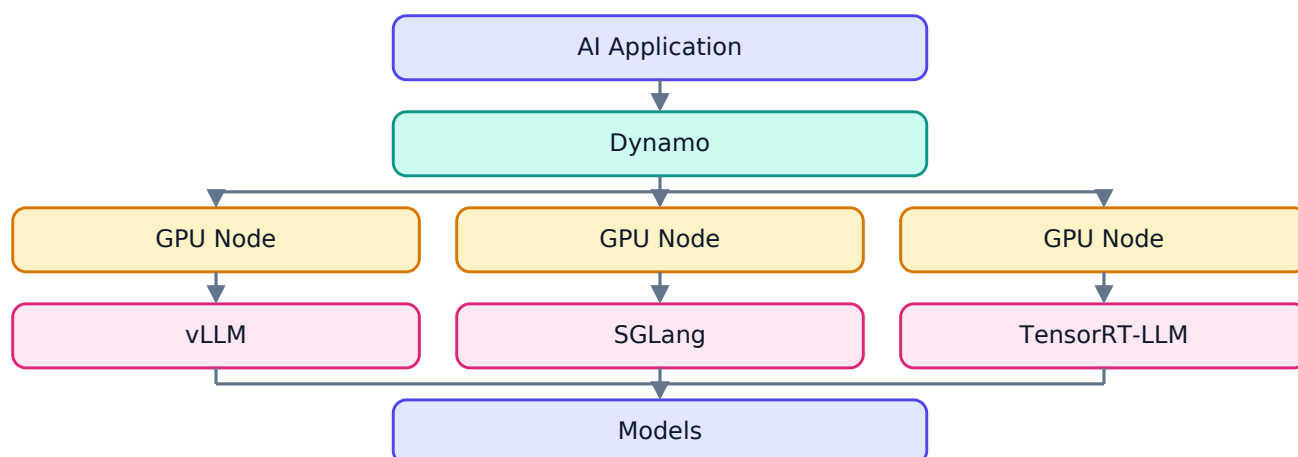
That's the real story.

21 Your final architecture slide

Make this the centerpiece of the presentation:



And for **large-scale infrastructure**:



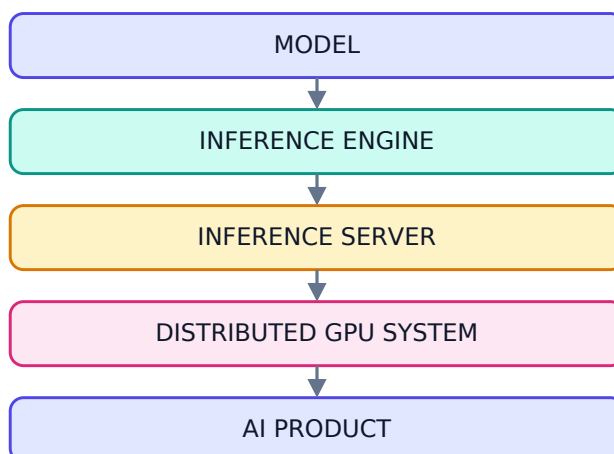
NVIDIA's current Dynamo architecture is specifically designed around distributed inference, request routing, KV reuse, scaling and separating inference phases. [6]

+ The killer conclusion

Don't finish with: "*Machine learning models are useful in various industries.*" That's the generic college ending. Finish with this idea:

In 2026, building an AI model is only half the job. The real engineering challenge is making that model fast, scalable, reliable and affordable for thousands or millions of users.

Then show:



And your final line:

"The future of AI isn't only about making smarter models. It's also about building better systems to serve them."

That gives you a presentation that connects **Machine Learning + Deep Learning + LLMs + GPUs + APIs + distributed systems + cloud + edge AI**, instead of just listing algorithms.

And the topic is genuinely current: as of September 2026, NVIDIA's Triton stack is already shipping current CUDA/TensorRT/vLLM components, vLLM is actively adding features around quantization,

speculative decoding and distributed serving, and NVIDIA Dynamo is positioned as a distributed inference layer for large-scale generative AI. [7]

§

Sources

- [1]** vLLM documentation
<https://docs.vllm.ai/en/stable/>

- [2]** NVIDIA Newsroom: NVIDIA Enters Production With Dynamo
<https://nvidianews.nvidia.com/news/dynamo-1-0>

- [3]** NVIDIA Dynamo: Scale and Serve Generative AI
<https://www.nvidia.com/en-in/ai/dynamo/>

- [4]** vLLM Quickstart
https://docs.vllm.ai/en/latest/getting_started/quickstart/

- [5]** NVIDIA Triton Inference Server Release Notes 26.08
<https://docs.nvidia.com/deeplearning/triton-inference-server/release-notes/rel-26-08.html>

- [6]** NVIDIA Dynamo: Overall Architecture
<https://docs.nvidia.com/dynamo/latest/knowledge-base/overview>

- [7]** NVIDIA Triton Inference Server Release Notes 26.07
<https://docs.nvidia.com/deeplearning/triton-inference-server/release-notes/rel-26-07.html>
